UNIVERSITÉ SAINT JEAN

Saint Jean Ingénieur — Sécurité du Cloud

RAPPORT DE PROJET

Stockage objet évolutif avec Swift

Application SYNCORA v2

Réalisé par	Encadrement
FOUDA DZOU AMBE Valdez **KAMDOUM TEUFENG Josette**	**Mr. NGUIMBUS Emmanuel** *Coordinateur pédagogique* Université Saint Jean *Année Académique 2025-2026*

TABLE DES MATIÈRES

Introduction
I. Cahier des charges
1.1 Contexte
1.2 Définition du problème
1.3 Objectifs du projet
1.4 Description des fonctionnalités
1.5 Ressources utilisées
II. Analyse
2.1 Besoins fonctionnels
2.2 Besoins non fonctionnels
2.3 Acteurs du système
2.4 Cas d'utilisation
III. Conception
3.1 Architecture générale
3.2 Composants principaux et modules
3.3 Diagramme de classes
3.4 Diagramme de séquence
3.5 Choix technologiques
IV. Réalisation
4.1 Structure du projet
4.2 Module d'initialisation — init.c
4.3 Module de surveillance — monitor.c
4.4 Module Swift — swift.c
4.5 Module d'orchestration — main.c
4.6 Déploiement systemd
4.7 Cycle de synchronisation
Conclusion

Introduction

Dans le cadre du module Sécurité du Cloud à l'Université Saint Jean, le présent rapport de projet documente l'intégralité du travail réalisé autour de la conception et de la réalisation de SYNCORA v2, une solution de stockage objet évolutif basée sur OpenStack Swift.

Ce rapport constitue le livrable final du projet et couvre l'ensemble du cycle de développement logiciel : depuis la définition des exigences dans le cahier des charges, l'analyse fonctionnelle et les cas d'utilisation, la conception architecturale avec les diagrammes UML, jusqu'à la réalisation technique de l'application.

SYNCORA v2 répond à un besoin concret : offrir aux utilisateurs Linux une alternative libre et ouverte aux clients de synchronisation cloud propriétaires comme OneDrive, en s'appuyant sur la robustesse et la scalabilité d'OpenStack Swift comme backend de stockage distribué. La solution développée se compose d'un démon de synchronisation écrit en langage C, déployé comme service système via systemd.

Périmètre du rapport

Ce rapport couvre : le cahier des charges, l'analyse (cas d'utilisation), la conception (diagrammes de classes et de séquence) et la réalisation du démon de synchronisation C (SYNCORA v2).

I. Cahier des charges

1.1 Contexte

La gestion des données numériques connaît une croissance exponentielle, imposant des solutions de stockage évolutives, fiables et hautement performantes. Les systèmes traditionnels basés sur le stockage par fichiers ou par blocs montrent rapidement leurs limites en matière de scalabilité, de disponibilité et de coût d'exploitation. Ces solutions peinent à s'adapter aux exigences actuelles des environnements numériques modernes.

C'est dans ce contexte que le stockage objet, et plus particulièrement la solution OpenStack Swift, s'impose comme une alternative innovante. OpenStack Swift est une plateforme de stockage distribuée conçue pour gérer de très grandes quantités de données non structurées tout en offrant une tolérance aux pannes élevée et une scalabilité horizontale illimitée. Sous Linux, il n'existe pas d'équivalent natif à OneDrive offrant une expérience de synchronisation transparente et automatique.

1.2 Définition du problème

L'explosion des données numériques nécessite des solutions de stockage scalables et fiables, particulièrement sous Linux où les alternatives à OneDrive font défaut. OpenStack Swift excelle en scalabilité horizontale et haute disponibilité via stockage objet distribué, mais son interface technique reste inaccessible aux utilisateurs non-experts.

Les lacunes identifiées de l'écosystème existant sont les suivantes :

Lacune identifiée	Impact utilisateur
Interface uniquement technique (API REST, CLI)	Inaccessible aux utilisateurs non-experts, nécessite des connaissances OpenStack avancées
Absence de client natif Linux	Aucune synchronisation automatique et transparente des fichiers sous Linux
Pas de sync bidirectionnelle temps réel	Impossibilité de maintenir la cohérence entre appareils sans intervention manuelle
Interface web Horizon non ergonomique	Expérience utilisateur dégradée, navigation fastidieuse
Verrou propriétaire Windows	Les utilisateurs Linux sont contraints de dépendre de solutions Windows non adaptées

1.3 Objectifs du projet

L'objectif principal est de développer une application capable d'automatiser la synchronisation des données vers OpenStack Swift. Les objectifs spécifiques sont les suivants :

1. Mettre en place une infrastructure Swift fonctionnelle et distribuée pour le stockage d'objets.

2. Développer une application de synchronisation automatique permettant de maintenir la cohérence entre les données locales et le cloud Swift.
3. Déployer l'application comme service système persistant via systemd pour une synchronisation transparente et continue.
4. Tester et valider le bon fonctionnement du système (sauvegarde, synchronisation bidirectionnelle, basculement en cas de suppression).

1.4 Description des fonctionnalités

Les fonctionnalités core de la solution développée sont les suivantes :

Fonctionnalité	Description
Synchronisation automatique locale ↔ cloud	Tout fichier ajouté, modifié ou supprimé dans les dossiers surveillés est répercuté automatiquement sur Swift dans un délai maximal de 5 secondes
Détection intelligente des changements	Utilisation du timestamp st_mtime et du flag --changed (MD5) pour ne transférer que les fichiers réellement modifiés
Gestion des conteneurs comme dossiers	Chaque conteneur Swift est représenté comme un sous-dossier dans ~/Documents/Syncora/, rendant le stockage cloud transparent
Synchronisation bidirectionnelle	Les nouveaux conteneurs créés sur Swift sont automatiquement reflétés localement, et inversement
Service système persistant	Déploiement via systemd avec démarrage automatique au boot et redémarrage en cas d'erreur
Journalisation native	Tous les événements de synchronisation sont journalisés via journalctl, permettant un diagnostic complet

1.5 Ressources utilisées

Ressources matérielles

Ressource	Description	Coût estimé
05 Machines Virtuelles	Cluster OpenStack Swift (nœuds de stockage, proxy, Keystone)	300 000 FCFA (6 mois)
Connexion Internet haut débit	Accès réseau pour les transferts de données	250 000 FCFA (10 mois)
10 Ordinateurs Portables	12 Go RAM, 256 Go SSD, 2.1 GHz	2 500 000 FCFA

Ressources logicielles

Logiciel	Rôle	Coût
OpenStack Swift	Service de stockage objet distribué	0
OpenStack Keystone	Service d'authentification et d'autorisation	0
GCC / Make	Compilation du démon C	0
Git / GitHub	Gestion de versions et collaboration	0
Visual Studio Code	Environnement de développement principal	0

II. Analyse

2.1 Besoins fonctionnels

Les besoins fonctionnels du système visent à reproduire et surpasser l'expérience OneDrive sous Linux. Le système doit assurer :

- **Un stockage et une gestion de données non structurées : le système doit pouvoir stocker et organiser efficacement de grandes quantités de données non structurées (images, vidéos, documents, etc.) sous forme d'objets avec métadonnées associées, géré nativement par Swift.**
- **Une évolutivité horizontale : la solution doit permettre d'ajouter facilement des nœuds de stockage pour augmenter la capacité sans interruption de service, garantissant une croissance continue et transparente.**
- **Une haute disponibilité et tolérance aux pannes : les données doivent être répliquées automatiquement sur plusieurs nœuds ou centres de données, assurant ainsi une continuité de service même en cas de défaillance matérielle.**
- **Une synchronisation et cohérence des données : le système doit permettre la synchronisation automatique des données locales entre différents appareils et assurer la cohérence des fichiers malgré les modifications simultanées réalisées en local.**

2.2 Besoins non fonctionnels

Les besoins non fonctionnels garantissent la qualité globale du service :

Critère	Exigence
Scalabilité	La capacité de stockage et la performance doivent pouvoir croître de manière linéaire avec l'ajout de nouvelles ressources sans perte de qualité de service.
Disponibilité élevée	Le service doit rester accessible en permanence, avec une tolérance aux pannes grâce à la redondance et la réplication des données.
Sécurité des données	Il est impératif d'assurer la confidentialité, l'intégrité et la protection des données, notamment via le chiffrement, les contrôles d'accès et la gestion des identités (Keystone).
Fiabilité et résilience	Le système doit être capable de résister aux défaillances matérielles ou logicielles sans perte de données ni interruption durable de service.
Interopérabilité	La solution doit pouvoir s'intégrer facilement avec les systèmes existants via des API standard RESTful et supporter différents protocoles de communication.
Expérience utilisateur	Malgré la complexité du système en arrière-plan, l'accès et l'utilisation des données doivent être simples, intuitifs et transparents pour les utilisateurs finaux.

2.3 Acteurs du système

Acteurs internes

Acteur	Rôles
Utilisateur	• Installer le client Swift sur sa machine • Activer le versioning • Modifier les fichiers • Se connecter / déconnecter sur la plateforme • Créer des fichiers • Restaurer une version antérieure du stockage
Client système (démon)	• Mettre à jour les fichiers • Supprimer des fichiers • Authentification au serveur OpenStack • Surveiller les modifications en temps quasi-réel

Acteurs externes

Acteur	Rôles
Plateforme	• Appels d'API vers OpenStack Swift • Visualisation des différentes versions
OpenStack Swift	• Réplication des données sur plusieurs nœuds • Versioning des données • Sauvegarde et stockage des objets

2.4 Cas d'utilisation

Les cas d'utilisation suivants décrivent les interactions entre les acteurs et le système SYNCORA.

CU-01 : Authentification au serveur OpenStack

Titre	Authentification au serveur OpenStack
Objectif	Permettre au client Swift de s'authentifier afin d'accéder aux services de stockage objet.
Acteurs	Client système (démon), Plateforme OpenStack (Keystone)
Préconditions	Le client possède des identifiants valides. Les services d'authentification Keystone sont disponibles.
Post-conditions	Un jeton d'authentification (X-Auth-Token) est délivré. Le client peut accéder aux API Swift.
Scénario nominal	• 1. Le client fournit ses identifiants (OS_USERNAME, OS_PASSWORD, OS_PROJECT_NAME). • 2. Keystone vérifie les informations d'authentification. • 3. Un jeton d'authentification est généré et retourné. • 4. Le jeton est transmis au client Swift pour les requêtes suivantes.

Scénarios alternatifs	• A1 : Identifiants invalides → accès refusé, message d'erreur retourné. • A2 : Service d'authentification indisponible → échec de la connexion, retry.

CU-02 : Installer le client Swift (démon SYNCORA)

Titre	Installer le client Swift
Objectif	Permettre à l'utilisateur d'installer et configurer le démon SYNCORA pour la synchronisation automatique des fichiers entre sa machine locale et la plateforme cloud.
Acteurs	Utilisateur, Client système, Plateforme
Préconditions	• L'utilisateur dispose d'une machine Linux compatible. • L'utilisateur a les droits administrateur nécessaires. • L'utilisateur dispose d'un compte valide sur la plateforme OpenStack. • La connexion internet est disponible.
Post-conditions	• Le démon syncora est installé dans /usr/local/bin/. • Le service systemd syncora.service est activé. • L'utilisateur peut sélectionner les dossiers à synchroniser via ~/Documents/Syncora/.
Scénario nominal	• 1. L'utilisateur clone le dépôt Git du projet. • 2. L'utilisateur compile le projet avec make clean && make. • 3. L'utilisateur copie le binaire dans /usr/local/bin/. • 4. L'utilisateur configure le fichier syncora.service avec ses credentials OpenStack. • 5. L'utilisateur active et démarre le service avec systemctl. • 6. Le démon démarre et crée automatiquement ~/Documents/Syncora/. • 7. La synchronisation initiale avec Swift est lancée.
Scénarios alternatifs	• A1 : Compilation échoue → vérifier les dépendances GCC et python3-swiftclient. • A2 : Connexion au controller échoue → vérifier /etc/hosts et les variables OS_*.

CU-03 : Mettre à jour les fichiers (sync automatique)

Titre	Mettre à jour les fichiers
Objectif	Permettre au client Swift de modifier un fichier déjà stocké en envoyant automatiquement une nouvelle version vers Swift.
Acteurs	Client système (démon), OpenStack Swift
Préconditions	• Le client est authentifié (token valide). • Le fichier existe dans le stockage Swift. • Le service syncora est en cours d'exécution.

Post-conditions	• Une nouvelle version du fichier est stockée sur Swift. • Le contenu distant est identique au contenu local.
Scénario nominal	• 1. L'utilisateur modifie un fichier dans ~/Documents/Syncora/conteneur/. • 2. Le démon détecte la modification via la comparaison de st_mtime lors du scan suivant (≤5s). • 3. Le démon appelle swift_upload() avec le flag --changed. • 4. La CLI Swift compare le MD5 local avec l'objet distant. • 5. Si différent, le fichier est transféré vers Swift. • 6. Le log [CLOUD] Upload confirme le succès.
Scénarios alternatifs	• A1 : Fichier non modifié (MD5 identique) → le flag --changed annule l'upload, aucun transfert. • A2 : Problème réseau → mise à jour reportée au prochain cycle.

CU-04 : Suppression de fichiers

Titre	Suppression de fichiers
Objectif	Permettre au client Swift de supprimer un objet stocké sur Swift lorsqu'un fichier local est supprimé.
Acteurs	Client système (démon), OpenStack Swift
Préconditions	• Le client est authentifié. • Le fichier existe dans Swift.
Post-conditions	• Le fichier est supprimé du cluster Swift. • L'opération est journalisée via journalctl.
Scénario nominal	• 1. L'utilisateur supprime un fichier de ~/Documents/Syncora/conteneur/. • 2. Au cycle suivant, le démon ne retrouve plus le fichier dans le scan courant. • 3. Il détecte l'absence et identifie le fichier comme [DEL]. • 4. swift_delete() est appelé pour supprimer l'objet sur Swift. • 5. Le log [CLOUD] Delete confirme l'opération.
Scénarios alternatifs	• A1 : Fichier inexistant sur Swift → message d'erreur journalisé, opération ignorée. • A2 : Suppression non autorisée → accès refusé par Keystone.

CU-05 : Création d'un fichier

Titre	Création d'un fichier
Objectif	Uploader automatiquement sur Swift tout nouveau fichier créé dans un dossier surveillé.
Acteurs	Utilisateur, Client système (démon), OpenStack Swift
Préconditions	• Le service SYNCORA est actif.

	• Le dossier de destination est un dossier surveillé (sous ~/Documents/Syncora/).
Post-conditions	• Le fichier est uploadé sur Swift. • Il apparaît dans le conteneur Swift correspondant.
Scénario nominal	• 1. L'utilisateur crée ou copie un fichier dans ~/Documents/Syncora/conteneur/. • 2. Le démon détecte le nouveau fichier (absent de l'état précédent) → [NEW]. • 3. swift_upload() est appelé avec le chemin du fichier. • 4. L'objet est créé sur Swift avec ses métadonnées. • 5. Le log [CLOUD] Upload confirme le succès.
Scénarios alternatifs	• A1 : Fichier trop volumineux → transfert lent, journalisé. • A2 : Perte de connexion → upload reporté au cycle suivant.

CU-06 : Sauvegarde des données par Swift

Titre	Sauvegarde des données
Objectif	Stocker durablement les données envoyées par le client sur les nœuds du cluster Swift.
Acteurs	OpenStack Swift (serveur)
Préconditions	• Swift est opérationnel. • Les nœuds de stockage sont disponibles.
Post-conditions	• Les données sont stockées et répliquées. • Les métadonnées sont enregistrées.
Scénario nominal	• 1. Swift reçoit la requête PUT avec les données de l'objet. • 2. Les objets sont distribués sur les nœuds selon le consistent hashing. • 3. La réplication (3 copies par défaut) est effectuée. • 4. La sauvegarde est confirmée avec un code HTTP 201.
Scénarios alternatifs	• A1 : Disque plein → sauvegarde échouée, erreur 507 retournée. • A2 : Nœud indisponible → redirection automatique vers un nœud de remplacement.

III. Conception

3.1 Architecture générale

SYNCORA repose sur une architecture client-serveur distribuée. Le client est un démon C installé sur la machine Linux de l'utilisateur. Le serveur est un cluster OpenStack Swift gérant le stockage distribué. L'authentification est déléguée à Keystone, le service d'identité standard d'OpenStack.

```
ARCHITECTURE GLOBALE DE SYNCORA v2

┌──────────────────────────────────────────────────────────────────────────────────────┐
│          Machine Linux (utilisateur)                                                 │
│                                                                                      │
│   ~/Documents/Syncora/                                                               │
│   ├── conteneur_A/   (fichier1.txt, image.png …)                                     │
│   └── conteneur_B/   (doc.pdf …)                                                     │
│              ↕ surveillance & sync                                                   │
│    [ syncora — démon C — service systemd ]                                           │
└──────────────────────────────────────────────────────────────────────────────────────┘
                   │  swift CLI (API REST)
                   ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│          Cluster OpenStack Swift                                                     │
│   ├── Keystone     (authentification — port 5000)                                    │
│   ├── Swift Proxy  (point d'accès — port 8080)                                       │
│   ├── conteneur_A  (objets + métadonnées répliqués)                                  │
│   └── conteneur_B  (objets + métadonnées répliqués)                                  │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

3.2 Composants principaux et modules

Le système SYNCORA se décompose en quatre modules C distincts, chacun portant une responsabilité clairement définie :

Module	Fichier	Rôle
Initialisation	init.c / init.h	Crée le répertoire ~/Documents/Syncora/ et les sous-dossiers correspondant aux conteneurs Swift existants au démarrage. Point d'entrée de la phase d'initialisation.

Surveillance	monitor.c / monitor.h	Parcourt chaque dossier surveillé, compare l'état courant avec l'état mémorisé dans FolderState, et détecte les créations (NEW), modifications (MODIF via st_mtime) et suppressions (DEL).
Client Swift	swift.c / swift.h	Encapsule les opérations Swift : swift_upload() avec flag --changed, swift_delete() pour les objets, swift_delete_container() pour les conteneurs entiers.
Orchestration	main.c	Boucle principale de synchronisation en 4 étapes ordonnées, gestion dynamique des FolderState, cycle de 5 secondes.

3.3 Diagramme de classes

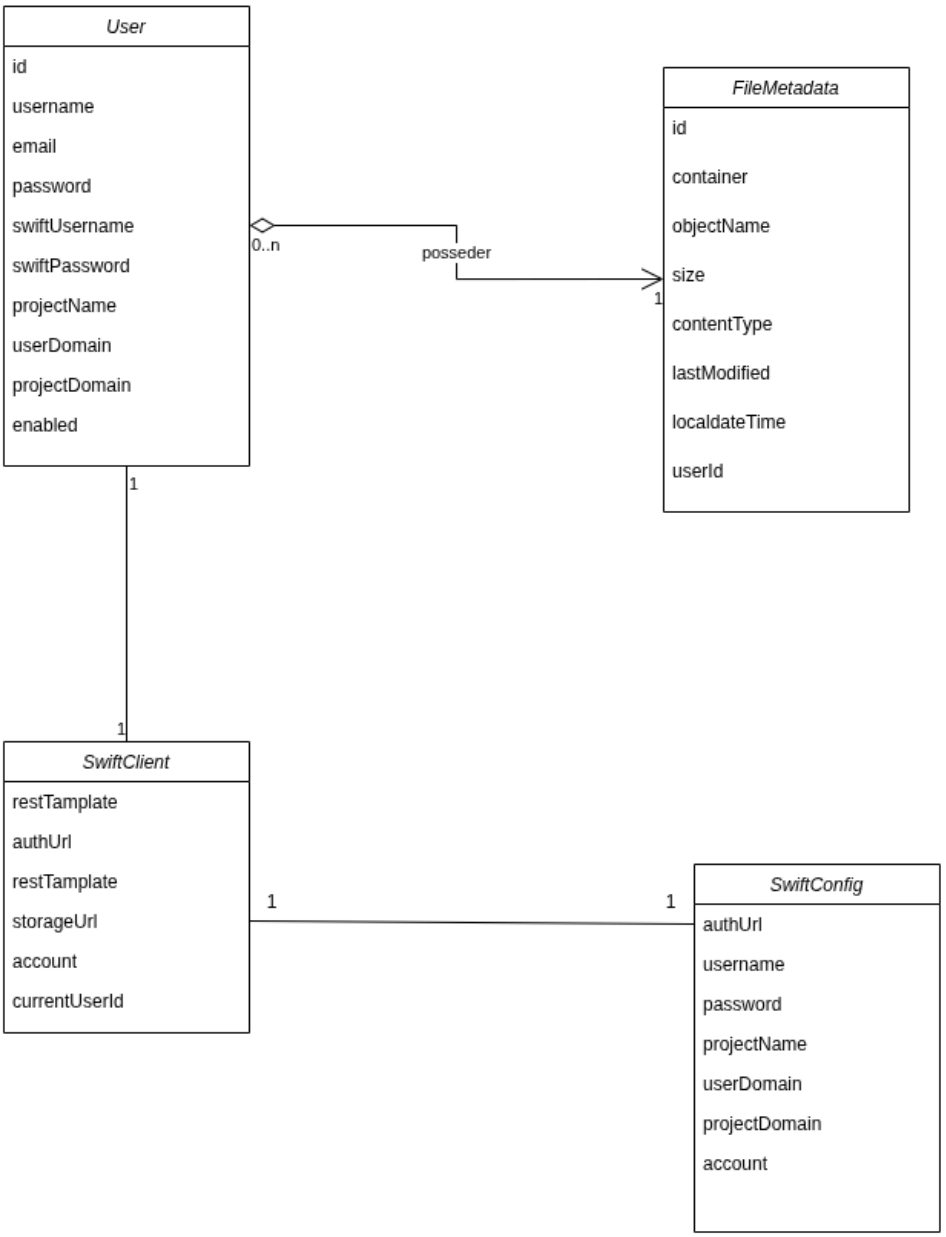
Le diagramme de classes présente les entités métier du système et leurs relations. Conformément au cahier de conception, les classes principales sont :

Classe	Attributs principaux	Rôle
User	• id : Long (PK) • username : String (unique) • email : String (unique) • password : String (BCrypt) • swiftUsername : String • swiftPassword : String (BCrypt) • projectName : String • enabled : boolean • createdAt : LocalDateTime	Stocke les identifiants et credentials OpenStack Swift de chaque utilisateur. Point central de l'authentification et de l'isolation des données.
FileMetadata	• id : Long (PK) • container : String • objectName : String • size : Long • contentType : String • lastModified : LocalDateTime • userId : Long (FK)	Persiste localement les métadonnées des fichiers synchronisés. Permet la consultation de l'historique sans requête Swift.
SwiftClient	• restTemplate : RestTemplate • authUrl : String • authToken : String • storageUrl : String • account : String • currentUserId : Long	Exécute les appels HTTPS REST vers les APIs OpenStack Swift avec authentification par utilisateur. Interface principale avec le cluster.
SwiftConfig	• authUrl : String • username : String	Fournit la configuration globale OpenStack. Contient les paramètres de connexion au cluster Swift.

	• password : String • projectName : String • userDomain : String • projectDomain : String • account : String	
FolderState	• folder_name : char[1024] • previous_files : file_info[1024] • previous_count : int	Mémorise l'état précédent d'un dossier surveillé (liste des fichiers + timestamps). Permet la détection des changements par différence d'état.
file_info	• name : char[256] • last_modified : time_t	Représente un fichier dans l'état mémorisé d'un dossier. Le timestamp last_modified (st_mtime) est la clé de détection des modifications.

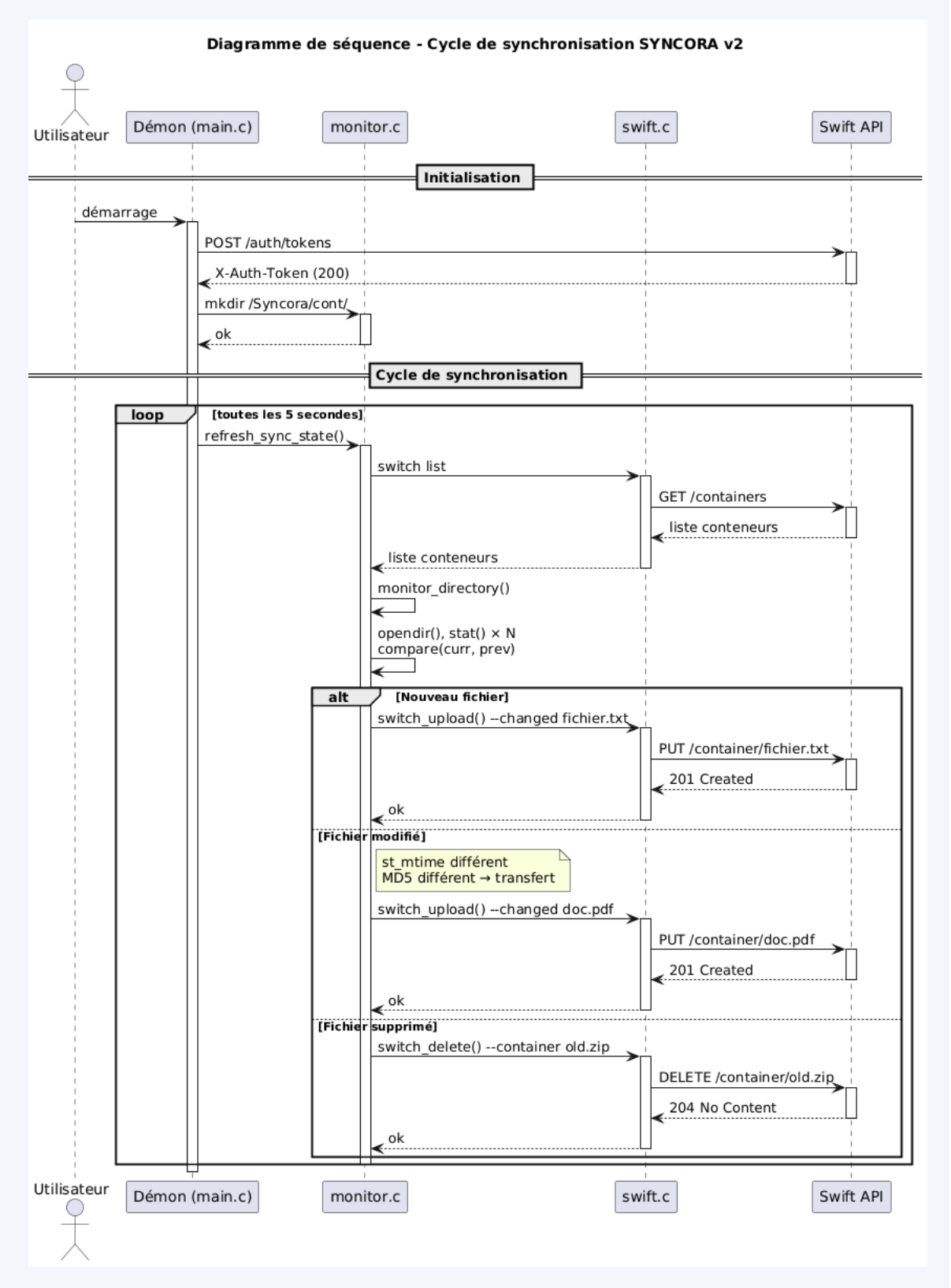
Relations entre classes :

Source	Type	Cible / Cardinalité
User	Composition 1–N	FileMetadata (un utilisateur possède plusieurs fichiers)
SwiftClient	Dépendance	SwiftConfig (utilise la configuration pour s'authentifier)
FolderState	Agrégation 1–N	file_info (contient jusqu'à 1024 entrées de fichiers)
main (orchestrateur)	Association 1–N	FolderState (gère jusqu'à 100 conteneurs surveillés)



3.4 Diagramme de séquence

Le diagramme de séquence ci-dessous représente le flux d'interaction entre les composants lors du cycle de synchronisation principal.



3.5 Choix technologiques

Les choix technologiques ont été opérés en tenant compte des besoins fonctionnels et non fonctionnels identifiés lors de l'analyse :

Composant	Technologie	Justification
Démon de sync	Langage C (GCC)	Légèreté, accès direct aux API POSIX (stat, opendir, mkdir), performance optimale pour un service système Linux permanent.
Détection de changements	stat() / st_mtime	API POSIX native, fiable et portable. Même technique que rsync. Alternative envisagée : inotify (amélioration future).
Interface CLI Swift	python3-swiftclient	Client officiel OpenStack Swift, disponible nativement sous Linux, gère le cycle de vie du token Keystone automatiquement.
Stockage cloud	OpenStack Swift	Scalabilité horizontale illimitée, tolérance aux pannes par réplication, API REST standard, open source.
Authentification	Keystone v3	Standard OpenStack, gestion des tokens et des projets, injection sécurisée via variables d'environnement systemd.
Déploiement service	systemd	Intégration native Linux, démarrage auto, restart automatique, journalisation via journalctl, aucun code de log à écrire.
Versionning code	Git / GitHub	Standard de facto pour le versioning, collaboration, traçabilité des modifications, intégration CI/CD.
IDE	Visual Studio Code	Léger, extensible, intégration native Git/Docker, support du C via l'extension C/C++ Microsoft.

IV. Réalisation

4.1 Structure du projet

Le projet livré est organisé selon l'arborescence suivante :

```
Stockage-objet-evolutif-avec-Swift-main/
├── LICENSE
├── README.md                    ← Documentation d'installation et
d'utilisation
├── config/
│   ├── demo-openrc              ← Variables d'environnement OpenStack
(exemple)
│   └── syncora.service.sample   ← Unité systemd à configurer
└── PROJET/
    └── C_syncora_v2/            ← Code source du démon C
        ├── main.c               ← Orchestration et boucle principale
        ├── init.c               ← Initialisation des conteneurs
        ├── init.h
        ├── monitor.c            ← Surveillance des répertoires
        ├── monitor.h            ← Structures FolderState et file_info
        ├── swift.c              ← Opérations Swift (upload, delete)
        ├── swift.h
        └── makefile             ← Compilation du projet
```

4.2 Module d'initialisation — init.c

La fonction setup_containers() constitue le point de départ du service. Elle réalise trois opérations séquentielles au démarrage du démon :

5. Lecture de la variable d'environnement HOME pour localiser le répertoire personnel de l'utilisateur.
6. Création du répertoire racine ~/Documents/Syncora/ via mkdir() si celui-ci n'existe pas encore.
7. Exécution de swift list via popen() pour récupérer la liste des conteneurs existants sur le cluster Swift, puis création d'un sous-dossier local pour chacun d'eux.

```
// Extrait init.c — setup_containers()
void setup_containers() {
    char *home = getenv("HOME");
    snprintf(base_path, sizeof(base_path), "%s/Documents/Syncora", home);
    mkdir(base_path, 0755);

    fp = popen("swift list", "r");
    while (fgets(container_name, sizeof(container_name), fp)) {
        container_name[strcspn(container_name, "\n")] = 0;
        // Crée le dossier local pour chaque conteneur Swift existant
        snprintf(full_folder_path, ..., "%s/%s", base_path,
container_name);
        if (stat(full_folder_path, &st) == -1)
            mkdir(full_folder_path, 0755);
```

```
    }
    pclose(fp);
}
```

4.3 Module de surveillance — monitor.c

La fonction monitor_directory() est le cœur de la détection des changements. Elle parcourt un répertoire surveillé et compare son état courant avec l'état mémorisé dans la structure FolderState lors du cycle précédent.

Les trois types d'événements détectés sont :

- **[NEW] — Fichier présent dans le scan courant mais absent de l'état précédent : déclenchement d'un upload.**
- **[MODIF] — Fichier présent dans les deux états mais avec un timestamp st_mtime différent : déclenchement d'un re-upload avec vérification MD5.**
- **[DEL] — Fichier présent dans l'état précédent mais absent du scan courant : déclenchement d'une suppression sur Swift.**

```
// Détection de modification — monitor.c
if (current_files[i].last_modified !=
state->previous_files[j].last_modified) {
    printf("[MODIF] %s\n", current_files[i].name);
    swift_upload(container_name, full_file_path);
}

// Détection de suppression — monitor.c
if (!found) {
    printf("[DEL] %s\n", state->previous_files[i].name);
    swift_delete(container_name, state->previous_files[i].name);
}
```

Les structures de données utilisées sont définies dans monitor.h :

```
// monitor.h — structures de données
#define MAX_FILES      1024   // Fichiers max par conteneur
#define MAX_CONTAINERS  100   // Conteneurs max surveillés

typedef struct {
    char   name[256];
    time_t last_modified;
} file_info;

typedef struct {
    char      folder_name[1024];
    file_info previous_files[MAX_FILES];
    int       previous_count;
} FolderState;
```

4.4 Module Swift — swift.c

Ce module encapsule les trois opérations d'interaction avec le cluster Swift. L'ensemble des appels est réalisé via la swift CLI (python3-swiftclient), qui gère le cycle de vie du token Keystone automatiquement à partir des variables d'environnement OS_*.

Fonction	Commande exécutée	Particularité
swift_upload()	swift upload <cont> '<path>' --changed	Le flag --changed compare le MD5 local avec l'objet Swift avant transfert, évitant les uploads redondants.
swift_delete()	swift delete <cont> '<file>'	Suppression d'un objet individuel dans un conteneur.
swift_delete_container()	swift delete <cont>	Suppression d'un conteneur entier lors de la suppression du dossier local correspondant.

```
// swift.c — swift_upload()
void swift_upload(const char *container, const char *filepath) {
    char command[4096];
    char *filename = strrchr(filepath, '/');
    filename = (filename) ? filename + 1 : (char *)filepath;

    snprintf(command, sizeof(command),
        "swift upload %s '%s' --object-name '%s' --changed",
        container, filepath, filename);

    printf("[CLOUD] Upload: %s/%s\n", container, filename);
    system(command);
}
```

4.5 Module d'orchestration — main.c

La fonction main() orchestre l'ensemble du processus. Après l'initialisation via setup_containers(), elle entre dans une boucle infinie composée de deux appels séquentiels : refresh_sync_state() puis monitor_directory() pour chaque conteneur surveillé, suivis d'un sleep(5).

La fonction refresh_sync_state() exécute le cycle de synchronisation en 4 étapes ordonnées, dont l'ordre est critique pour la cohérence des données :

Étape	Opération	Description
1	Suppression prioritaire (local → cloud)	Si un dossier local a disparu, le conteneur Swift correspondant est supprimé EN PREMIER. Cette priorité évite les conflits lors des étapes suivantes.
2	Création locale (cloud → local)	Tout nouveau conteneur détecté sur Swift qui n'existe pas localement est créé comme sous-dossier dans ~/Documents/Syncora/.

3	Création distante (local → cloud)	Tout nouveau dossier créé localement par l'utilisateur déclenche la création du conteneur Swift correspondant via swift post.
4	Mise à jour de la liste FolderState	La liste des conteneurs surveillés est rafraîchie pour refléter l'état actuel, intégrant les créations et suppressions des étapes précédentes.

```
// main.c — boucle principale
int main() {
    char *home = getenv("HOME");
    setup_containers();

    FolderState *states = malloc(sizeof(FolderState) * MAX_CONTAINERS);
    int count = 0;

    printf("=== SYNCORA V2 : DYNAMIQUE ET BIDIRECTIONNEL ===\n");

    while (1) {
        refresh_sync_state(states, &count, home);  // 4 étapes sync
        for (int i = 0; i < count; i++)
            monitor_directory(&states[i]);          // surveillance
fichiers
        sleep(5);                                   // cycle 5 secondes
    }
    free(states);
    return 0;
}
```

4.6 Déploiement systemd

SYNCORA v2 est déployé comme un service système Linux géré par systemd. Le fichier de service syncora.service, fourni dans le dossier config/ du dépôt, configure le démon avec les variables d'environnement OpenStack et assure sa persistance.

```
[Unit]
Description=Service de Synchronisation Syncora Cloud (OpenStack)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=username                               # Utilisateur Linux
d'exécution
Group=username

# Variables d'authentification OpenStack (Keystone)
Environment="OS_AUTH_URL=http://controller:5000/v3"
Environment="OS_PROJECT_NAME=nom_du_projet"
Environment="OS_USERNAME=utilisateur"
Environment="OS_PASSWORD=mot_de_passe"
Environment="OS_PROJECT_DOMAIN_NAME=Default"
Environment="OS_USER_DOMAIN_NAME=Default"
```

```
Environment="OS_IDENTITY_API_VERSION=3"

WorkingDirectory=/chemin/vers/C_syncora_v2
ExecStart=/usr/local/bin/syncora
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Commandes d'activation et de supervision du service :

```
# Compilation et installation
make clean && make
sudo cp syncora /usr/local/bin/syncora

# Activation du service
sudo systemctl daemon-reload
sudo systemctl enable syncora.service
sudo systemctl start syncora.service

# Supervision en temps réel
journalctl -u syncora.service -f
systemctl status syncora.service
```

4.7 Cycle de synchronisation

Une fois le service démarré, l'application surveille en continu le répertoire ~/Documents/Syncora/. Le cycle de synchronisation de 5 secondes produit les événements journalisés suivants :

Événement log	Déclencheur	Action Swift exécutée
[INIT]	Démarrage du service	Création des dossiers locaux correspondant aux conteneurs Swift existants
[NEW]	Nouveau fichier détecté	swift upload <container> '<file>' --object-name '<name>' --changed
[CLOUD] Upload	Après NEW ou MODIF	Confirmation de l'upload avec chemin conteneur/fichier
[MODIF]	st_mtime modifié	swift upload ... --changed (vérification MD5 avant transfert)
[DEL]	Fichier absent du scan	swift delete <container> '<filename>'
[SYNC-IN]	Nouveau conteneur Swift distant	mkdir(~/Documents/Syncora/<container_name> , 0755)

Conclusion

Ce rapport de projet documente l'intégralité du cycle de développement de SYNCORA v2, depuis la définition des exigences jusqu'à la réalisation technique. Le projet a répondu à une problématique concrète : offrir aux utilisateurs Linux un client de synchronisation cloud libre, transparent et automatique, reposant sur la robustesse d'OpenStack Swift.

La solution développée — un démon C déployé comme service systemd — démontre qu'il est possible de construire un outil de synchronisation efficace et léger en s'appuyant sur les API POSIX natives de Linux et la swift CLI. Le cycle de synchronisation de 5 secondes, la détection des changements via st_mtime, et la gestion bidirectionnelle des conteneurs constituent le cœur fonctionnel de SYNCORA v2.

L'approche architecturale modulaire retenue — quatre modules C aux responsabilités distinctes — facilite la maintenance et l'évolution de la solution. Le déploiement via systemd garantit la persistance, la résilience et l'observabilité du service en production.

Bilan du projet

— Cahier des charges : fonctionnalités de synchronisation définies et implémentées
— Analyse : 6 cas d'utilisation documentés, acteurs et besoins formalisés
— Conception : diagrammes de classes et de séquence produits
— Réalisation : démon C fonctionnel (4 modules), service systemd opérationnel
— Technologies utilisées : C / GCC, OpenStack Swift, Keystone, systemd, Git, VSCode